Signal, Image and Video Processing (2025) 19:668
https://doi.org/10.1007/s11760-025-04217-1

ORIGINAL PAPER

Real-Time Image Dehazing via a Novel Parallel Architecture: An Empirical Comparison of Hardware Platforms

Avra Ghosh[1] · **Sumanta Bhowmic**[2] · **Sheli Sinha Chaudhuri**[1]

Received: 12 January 2025 / Revised: 5 April 2025 / Accepted: 20 April 2025 / Published online: 28 May 2025

Abstract
Atmospheric particles significantly degrade image quality, posing challenges for computer vision applications requiring high-contrast and clear images. To address this issue, a novel parallel architecture for real-time image dehazing is proposed, incorporating an optimized atmospheric light estimation module and an enhanced guided filter for transmission coefficient estimation, designed for hardware-efficient implementation. Extensive qualitative and quantitative comparisons were conducted across Raspberry Pi Model 4B, PYNQ Z2, and NVIDIA Jetson Nano, through which the proposed parallel architecture was shown to outperform existing techniques in both processing speed and dehazing quality. The PYNQ Z2 platform was found to deliver superior perceptual and structural performance across all evaluation metrics, including PSNR, SSIM, MS-SSIM, CORR, MSE, and LPIPS(d). The effectiveness of hardware-level, parallel dehazing for enhancing image visibility and system reliability under adverse conditions is thus demonstrated. The source code and datasets used in this study have been made publicly available to support reproducibility and further research.

Keywords Image Dehazing · Parallel Architecture · FPGA Implementation · Hardware Model

1 Introduction

Outdoor scene images often suffer from degradation due to adverse weather conditions, where atmospheric phenomena such as fog, haze and can be influenced by various factors such as mist, vog, dust, and industrial pollutants significantly reduce visibility. The presence of additional particles in the air causes light scattering, leading to lower contrast, faded colors, and decreased sharpness, making distant objects appear indistinct. This issue has drawn increasing attention in recent research due to its impact on image clarity and scene perception.

Various enhancement techniques, including histogram equalization, denoising, and lighting adjustments, have been employed to mitigate these issues. However, these methods sometimes lead to information loss, making it crucial to incorporate an effective haze removal process that enhances image visibility without compromising essential details. Histogram equalization, Retinex-based methods, and physical methods that compare numerous images to the hazy image in order to eliminate the haze are some of the ways available for dehazing an image. The single picture haze removal method is an additional strategy. To dehaze the image in this instance, single image haze reduction technique is employed.

To increase the use of these theories or programs in all contexts, the challenge is to implement them at the chip level in hardware logic. Realizing a high frame rate for excellent-quality photos is another difficulty. Our suggested approach seeks to resolve these problems and put them into practice at the hardware level. In this paper, a novel architecture is proposed for both hardware and software algorithms which will dehaze the image in real-time. Also the algorithm is compared in different hardware systems like Raspberry Pi Model-4B, Pynq Z2 board, NVIDIA Jetson NANO board and the shared the best result respect to qualitative and quantitative comparison.

✉ Avra Ghosh
avra.ece@gmail.com

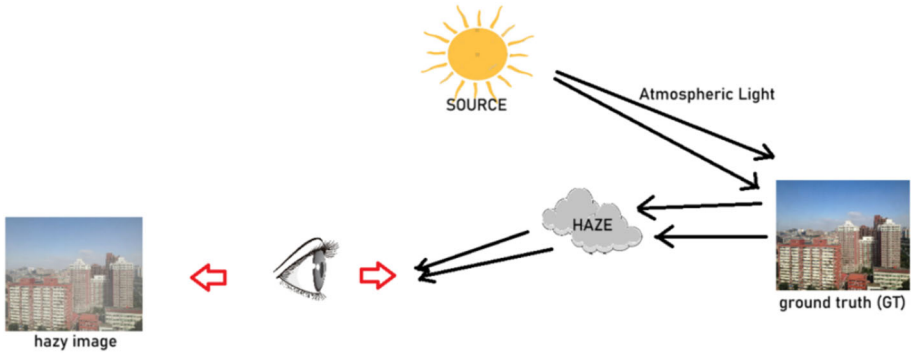
Sumanta Bhowmic
sumantabhowmick2004@gmail.com

Sheli Sinha Chaudhuri
sheli.sinha@jadavpuruniversity.in

[1] ETCE Department, Jadavpur University, Kolkata, India

[2] ECE Department, Institute of Engineering and Management, Kolkata, India

Fig. 1 Haze formation logic



1.1 Main Contributions

The key contributions of this paper are as follows:

- **Novel Parallel Architecture:** We propose a hardware-efficient parallel architecture that integrates an **optimized guided filter** for transmission coefficient estimation and an **enhanced atmospheric light estimation module**, improving both dehazing accuracy and computational efficiency.
- **Comprehensive Hardware Comparison:** We conduct a detailed evaluation across multiple hardware platforms, including Raspberry Pi Model 4B, Pynq Z2, and NVIDIA Jetson Nano, providing **quantitative performance metrics** and a clear discussion on the most suitable options for real-time applications.

This paper is organized into several key sections: it begins with a Literature Review, followed by a detailed Background that sets the foundation for the study. The proposed model and its architecture are then introduced, leading into a comprehensive analysis that includes both qualitative and quantitative evaluations, comparing the results with ground truth and existing methods. Finally, the Conclusion summarizes the findings and presents the final decision of the research.

2 Literature Review

Haze from suspended particles is a major contributor in the loss of image quality in computer vision technologies. Many techniques have been put forth to get rid of haze. In an effort to enhance the quality of grayscale photographs, Oakley et al. [5] employed a physical model in which depth information was obtained using radar calibration. After that, Narasimhan et al. [1] presented a technique for quickly obtaining depth information by comparing two images-one acquired in favorable weather conditions and the other in unfavorable conditions-using a polarizing filter. A comparable method was employed in Chen et al.'s study [8], although it necessitated the acquisition of previous images. A sunny and a foggy image were utilized as references. Later, Tan [10] offered a superior method that produces high-quality image dehaze. Unfortunately, the halo effects created by this method led to a loss of information in the dehazed photos and a degradation in image quality.

He [13] introduced a dehazing technique based on dark channel prior (DCP) around the same period, as will be discussed below. The procedure starts by estimating the dark channel before estimating atmospheric light, as shown in the flowchart. The transmission coefficient is then calculated using this approximation. After the scene is restored, the soft matting algorithm is applied to the image, producing the final dehazed picture. The problem with this technique, though, is that soft matting has a high temporal complexity, which makes the program execute slowly overall. Furthermore, problems occur in the dehazed image when the hazy photos contain a lot of white space or sky. Some obstacles still exist, despite the fact that many advancements based on the dark channel prior theory have produced higher-quality dehazed images.

Wang investigated a single pixel technique in a recent work [17], pointing out that noise can deteriorate the quality of dehazed photos. These days, dehazing techniques based on deep learning are producing amazing outcomes. Bolun Cai et al. [26], for example, presented a deep learning methodology that incorporates the dark channel prior in 2016, which represents a move away from conventional models and toward CNN-based techniques. Subsequently, in 2017 Boyi Li et al. [27] presented AOD-NET, a CNN architecture that is lightweight and specifically intended for image dehazing. In 2022, Yuda Song and colleagues [28] created a U-Net-based deep learning network to improve picture dehazing even more. In 2023, A. Ali et al. [29] introduced a light-invariant method for dehazing images taken at any time. H. Qin et. al. [32] proposed a deep learning-based image dehazing method leveraging the Zero-DCE approach to dynamically adjust image curves using unpaired data, achieving real-time performance at 1000

FPS on 2K resolution with enhanced clarity and brightness. Y. Wei [33] presents UMA-Net, a U-shaped multi-scale adaptive selection network that uses multi-scale adaptive and channel attention-guided fusion modules to effectively expand the receptive field and improve spatial and channel information, resulting in state-of-the-art image dehazing performance at low computational cost. However, implementing these deep learning methods on hardware poses significant challenges due to their complexity. Additionally, the space required to implement CNN-based methods is considerably higher compared to classical approaches. Recent advancements in real-world image dehazing focus on enhancing adaptability by incorporating imaging perspectives, structural modules, and training strategies [34]. A novel haze imaging model encapsulates multiple degradation factors, reducing the domain gap between synthetic and real-world images. Unpaired dehazing methods leverage density and depth decomposition, improving robustness by estimating scattering coefficients and scene depth [35]. A dual contrastive perceptual loss further refines dehazing quality and generalization. Adversarial deformation constraints help regularize solution space, minimizing dehazing errors while enhancing structural consistency [36]. These methods collectively improve dehazing performance on real-world datasets by addressing haze variation and feature dependencies.

All of these algorithms now need to be implemented on tiny chips, which is a difficult and complicated task. Hardware realization is particularly challenging for functions like airlight estimate, transmission coefficient calculation, and soft matting algorithm implementation. Yeu-Hoing Shiau, Hu G-Yu Yang, and Pei-Yin [16] presented an 11-stage pipelined hardware architecture for haze removal in 2013. The hardware implementation included mean and edge-preserving filters. In 2016, Bin Zhang and JiZhong Zhao [21] subsequently put up a 15-stage pipelined architecture for the elimination of haze. In comparison to the earlier architecture, this current one uses less hardware resources and offers higher computing performance.

Yeu-Hoing Shiau, Hu G-Yu Yang, and Pei-Yin [16] presented an 11-stage pipelined hardware architecture for haze removal in 2013. The hardware implementation included mean and edge-pr eserving filters. In 2016, Bin Zhang and JiZhong Zhao [21] subsequently put up a 15-stage pipelined architecture for the elimination of haze. In comparison to the earlier architecture, this current one uses less hardware resources and offers higher computing performance. To address the low time complexity and intricate hardware architectures of earlier approaches, Yu-Hsuan Lee and Bo Hua [22] suggested a hardware implementation methodology in 2016. Low Complexity Airlight Estimation (LAE) and Independent Transmission Rate Estimation (ITRE) are the two new fundamental methodologies they introduced. The heavy use of Look-Up Tables (LUTs) and overall hardware complexity make real-time implementation difficult, even if these techniques greatly reduce logic and time complexity. Furthermore, information loss and quality degradation result from the requirement to downsample the image for LAE calculation. Enhancements in both domains are required since real-time applications need a challenging trade-off between time complexity and quality. In 2020, A. Ghosh et al. [31] introduced a simulation-based single image dehazing model for image dehazing technology. Additionally, A. Ghosh et al. [30] proposed the haze parameter SATVAL, designed to determine which frames require dehazing to extract data, and implemented this in a Raspberry Pi model to create a real-time system. However, the methods mentioned above either rely solely on simulated results or fail to achieve the image quality necessary for real-time applications-a limitation that this research publication aims to address.

Deep learning approaches like CNNs and GANs have advanced image dehazing, though challenges remain in perceptual quality and generalization across hazy conditions [37]. Optimized 2D FIR filter architectures reduce complexity and enhance performance, but balancing low power, area, delay, and accuracy is still difficult [38]. While traditional and learning-based haze removal methods improve visibility, real-time hardware implementation remains a barrier for low-cost systems [39]. CNN-based models predicting transmission maps and atmospheric light achieve superior MSE, SSIM, and PSNR, outperforming traditional techniques [40].

3 Background

An equation for describing hazy images was first proposed by H. Koschmeider [9] in 1924, and it was subsequently examined and explored by M.C. Cartney [24] in 1976. The following equation illustrates how foggy images arise in inclement weather.

$$I(x) = J(x)t(x) + A(1 - t(x)) \tag{1}$$

The hazy image is represented by $I(x)$, the original dehazed image is represented by $J(x)$, the transmission coefficient, $t(x)$, specifies the percentage of light that can pass through the haze, and ambient light is represented by A. We are able to obtain the dehazed image by using this model. The number of unknowns in the equation presents the biggest difficulty. There are three unknowns that need to be calculated: A, $t(x)$, and $J(x)$. The sole known variable is $I(x)$, and calculating $J(x)$ is the ultimate objective. First, the ambient light A and the transmission coefficient $t(x)$ are estimated.

As was previously mentioned, Kaiming He proposed the Dark Channel Prior (DCP) approach in 2009 [13]. The dark

channel, which can be used as a mask, is calculated using this method. Below is the equation for the dark channel:

$$f^{dark}(x) = \min_{y \in \sigma(x)} \left\{ \min I^c(y) \right\} \tag{2}$$

where the square window of the minimum filter, centered at the x coordinate, is represented by. In this case, the RGB color channels Red, Green, Blue are represented by C. The ambient light, which is defined as the upper 0.1 % of the brightest pixels in the dark channel $J^{dark}(x)$, must next be computed. Following the determination of the ambient light, the transmission coefficient $t(x)$ is computed and shown as follows:

$$t(x) = 1 - w \cdot \min_{y \in \partial(x)} \left\{ \min_{c \in \{R,G,B\}} \frac{I^c(y)}{A^c} \right\} \tag{3}$$

Because eliminating all of the haze would cause the dehazed image to lose depth, the parameter w is used to keep a tiny amount of haze in the image to preserve a natural look. The atmospheric light is represented by A in several color channels. Modifications such as the Minimum Order Statistic Filter (MOSF) or the Guided Filter [14] can be used to enhance the transmission coefficient computation. The recovery of the original image is the next phase, and it is represented by the mathematical model that follows:

$$J(X) = \frac{I(X) - A}{max t(X), t_0} + A \tag{4}$$

where the lower bound t_0 is utilized to constrain $I(x)$. Scene recovery is possible by solving all of these equations together, although doing so requires performing intricate operations like sorting, averaging, matrix multiplication, and division.

4 Haze Removal Method

The algorithmic contributions are threefold: First, the Atmospheric Light estimation has been optimized to achieve faster results. Second, the guided filter for the Transmission coefficient has been enhanced to improve image quality. Lastly, a parallel architecture has been proposed to increase computational efficiency as shown in Fig 2.

4.1 Atmospheric Light Estimation

The following equation represents the *erosion operation* applied to each color channel c of an image. The variable $\text{minImg}_c(x, y)$ denotes the minimum value within a window W_k centered at the pixel position (x, y). This window slides over the image, and for each position (u, v) within the window, the pixel value at $(x + u, y + v, c)$ in the channel c is considered. The minimum of these values is assigned to the corresponding pixel in the eroded image:

$$\text{minImg}_c(x, y) = \min_{(u,v) \in W_k} H(x + u, y + v, c) \tag{5}$$

The airlight A_c for each channel c is estimated by finding the maximum value of the eroded image $\text{minImg}_c(x, y)$ across all pixel positions (x, y) in the image. This airlight value represents the highest intensity of the scattered light in the scene for the specific color channel:

$$A_c = \max_{(x,y)} \text{minImg}_c(x, y) \tag{6}$$

The final airlight vector A is formed by combining the airlight values A_c from all channels. This vector contains the airlight values for all channels C, typically representing the RGB channels in an image. The airlight vector is crucial for subsequent processing in image dehazing or enhancement:

$$A = [A_1, A_2, \ldots, A_C] \tag{7}$$

4.2 Transmission Guide

Mean Calculations:

- **Mean of** I:

$$\mu_I = \text{mean}(I) = \frac{1}{|\omega|} \sum_{i \in \omega} I(i) \tag{8}$$

The mean of the image I is denoted by μ_I. It is calculated by summing the values of $I(i)$ over all pixels i within the window ω and then dividing by the total number of pixels $|\omega|$ in that window.

- **Mean of** p:

$$\mu_p = \text{mean}(p) = \frac{1}{|\omega|} \sum_{i \in \omega} p(i) \tag{9}$$

The mean of the image p is denoted by μ_p. It is calculated similarly by averaging the pixel values $p(i)$ within the window ω.

- **Mean of** $I \times p$:

$$\mu_{Ip} = \text{mean}(I \times p) = \frac{1}{|\omega|} \sum_{i \in \omega} I(i) \times p(i) \tag{10}$$

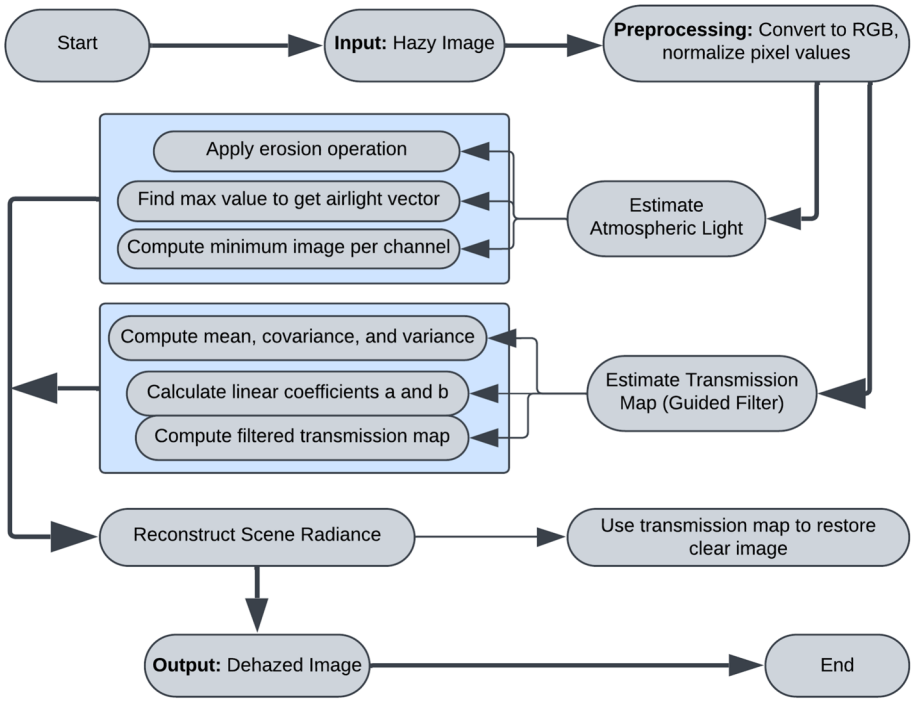


Fig. 2 Flow Diagram for Proposed Method

The mean of the product of images I and p is denoted by μ_{Ip}. It is obtained by first calculating the product of the corresponding pixel values $I(i) \times p(i)$ for all i in the window ω, summing these products, and then dividing by the number of pixels $|\omega|$ in that window.

Covariance and Variance Calculations:

- **Covariance between** I and p:

$$\sigma_{Ip} = \mu_{Ip} - \mu_I \times \mu_p \tag{11}$$

The covariance between the images I and p, denoted by σ_{Ip}, is calculated as the difference between the mean of the product of I and p (i.e., μ_{Ip}) and the product of their individual means μ_I and μ_p.

- **Variance of** I:

$$\sigma_I^2 = \mu_{II} - \mu_I^2 \tag{12}$$

The variance of the image I, denoted by σ_I^2, is calculated as the difference between the mean of the squared values of I (i.e., μ_{II}) and the square of the mean of I (i.e., μ_I^2). *Note:* μ_{II} represents the mean of the product of I with itself, i.e., $\mu_{II} = \text{mean}(I \times I)$.

3. **Linear Coefficients:**
- Coefficient a:

$$a = \frac{\sigma_{Ip}}{\sigma_I^2 + \epsilon} \tag{13}$$

- Coefficient b:

$$b = \mu_p - a \times \mu_I \tag{14}$$

Mean of Coefficients:

- **Mean of** a:

$$\mu_a = \text{mean}(a) \tag{15}$$

The mean of the coefficient a, denoted by μ_a, is calculated by averaging all the values of a across the relevant domain.

- **Mean of** b:

$$\mu_b = \text{mean}(b) \tag{16}$$

Similarly, the mean of the coefficient b, denoted by μ_b, is obtained by averaging all the values of b over the relevant domain.

Output Calculation (Filtered Image):

$$q(i) = \mu_a(i) \times I(i) + \mu_b(i) \tag{17}$$

The filtered output $q(i)$ at each pixel i is computed by multiplying the mean coefficient $\mu_a(i)$ with the input image intensity $I(i)$ at that pixel, and then adding the mean coefficient $\mu_b(i)$. This represents the basic formulation of the guided filter's output.

Summary Equation:
The guided filter's output $q(i)$ for each pixel i can be expressed as:

$$q(i) = \left(\frac{\text{mean}(I \times p) - \text{mean}(I) \times \text{mean}(p)}{\text{mean}(I \times I) - (\text{mean}(I))^2 + \epsilon}\right) \times I(i) + \left(\text{mean}(p) - \frac{\text{mean}(I \times p) - \text{mean}(I) \times \text{mean}(p)}{\text{mean}(I \times I) - (\text{mean}(I))^2 + \epsilon} \times \text{mean}(I)\right) \quad (18)$$

In this equation, the guided filter's output $q(i)$ is computed using a combination of mean values and pixel intensities. The first term represents the slope $\mu_a(i)$ derived from the covariance between I and p, normalized by the variance of I with a regularization term ϵ. The second term $\mu_b(i)$ is the intercept that adjusts the output based on the mean values of p and I.

4.3 Parallel Execution and Synchronization:

- **Atmospheric Light Estimation:**

$$A = \mathcal{A}(I, M, w) \quad (19)$$

The atmospheric light A is estimated using the function $\mathcal{A}$, which takes as input the image I, a mask M, and a weight w. This function outputs the estimated atmospheric light A.

- **Raw Transmission Estimation:**

$$\text{rawt}, \text{rawt}_p = \mathcal{T}(I, r) \quad (20)$$

The raw transmission maps, rawt and rawt_p, are computed using the function $\mathcal{T}$, which takes the image I and a radius parameter r as inputs.

- **Parallel Execution:**

$$\begin{aligned} \text{Parallel Execution:} \quad A &= \mathcal{A}(I, M, w), \\ (\text{rawt}, \text{rawt}_p) &= \mathcal{T}(I, r) \end{aligned} \quad (21)$$

During parallel execution, the atmospheric light A is estimated simultaneously with the computation of the raw transmission maps rawt and rawt_p. These operations are executed in parallel to improve efficiency.

- **After Synchronization:**

$$\text{After Synchronization:} \quad A \quad \text{and} \quad (\text{rawt}, \text{rawt}_p) \quad (22)$$

After parallel execution, the results A and $(\text{rawt}, \text{rawt}_p)$ are synchronized, meaning that both results are available and can be used for subsequent processing.

5 Hardware Description

5.1 Raspberry Pi

The need for small, single-chip digital processors, such as the Raspberry Pi, has increased recently. With the help of add-ons, the Raspberry Pi, a tiny single-board computer, may operate as an embedded Internet of things system. One of the newest versions, the Raspberry Pi 4 Model B, has 4GB of LPDDR4-2400 SDRAM and a quad-core Cortex-A72 (ARM v8) 64-bit processor powered by the Broadcom BCM2711 SoC, operating at 1.5GHz.

This model stands out thanks to its fast 64-bit quad-core processor, which offers great networking and multimedia performance. It offers up to 4GB of RAM, enables hardware video decoding up to 4Kp60, and supports multiple displays with resolutions up to 4K via dual micro-HDMI connectors. Gigabit Ethernet, USB 3.0, Bluetooth 5.0, dual-band 2.4/5.0 GHz wireless LAN, and Power over Ethernet (PoE) functionality (with an extra PoE HAT add-on) are among the connectivity features. The Raspberry Pi Model 4B is a great choice for effective real-time video processing because of these qualities. An SD card that is externally inserted loads the operating system. The detailed specification of Raspberry Pi Model 4B is given below. see website: **https://www.raspberrypi.com/**

The figure 3 shows an experimental configuration with a Raspberry Pi Model 4B at its center. The dual micro-HDMI connectors of the Raspberry Pi 4 Model B are used to connect the device to external monitors, demonstrating its ability to provide output with excellent resolution. A keyboard, mouse, and optionally external storage devices are linked as peripherals to improve functionality, and a camera module is connected to record video input in real time. To ensure stable functioning, the configuration also involves a cooling solution, like a heat sink or fan, and a power supply, usually a USB-C adapter. This setup makes use of the Raspberry Pi's potent computing capacity to process videos and do other computational activities quickly and effectively.

5.2 PYNQ Z2

For the Xilinx Zynq-7000 SoC (System on Chip), which combines a dual-core ARM Cortex-A9 CPU with programmable logic (FPGA), the PYNQ-Z2 board is a flexible development platform. Because it allows hardware-software co-design, this board is well-liked and perfect for embedded systems, Internet of Things, and machine learning applications.

A wide range of connectivity choices, such as Ethernet, USB, HDMI, audio, and Pmod connectors, enable broad interaction with sensors, displays, and other peripherals on the PYNQ-Z2 board. A microSD card port for storage, sev-

Fig. 3 Experiment Setup with Raspberry Pi Model 4B

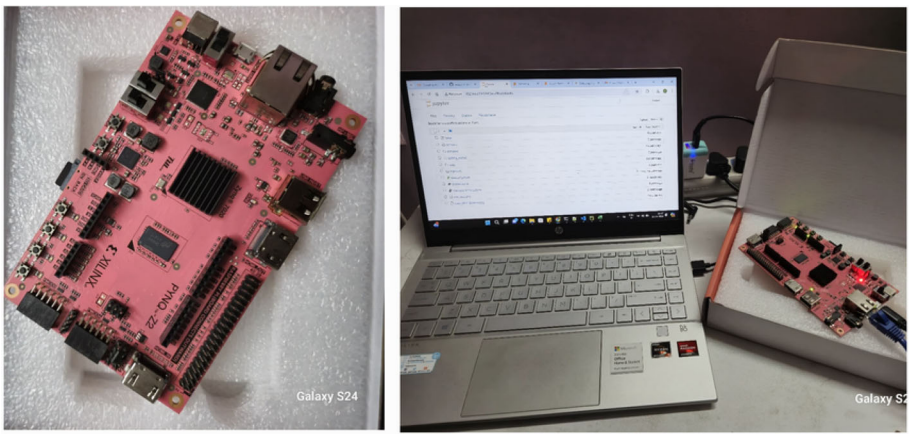


Fig. 4 PYNQ Z2 BOARD

eral GPIO pins for general-purpose input/output, and 512MB of DDR3 RAM are also included.

Support for the PYNQ framework, which enables users to program the FPGA using Python and increases accessibility to FPGA development for software engineers and data scientists, is one of the PYNQ-Z2's primary advantages. The board has Pmod connections and GPIO pins for bespoke hardware projects, and its HDMI ports facilitate video processing applications. The PYNQ-Z2 is a popular tool for investigating the combination of hardware and software on a single platform in academic research, prototyping, and instructional settings.

On the Pynq Z2 board, image dehazing is executed using FPGA-accelerated hardware blocks. The input image is transferred from the PS (Processing System) to the PL (Programmable Logic) via AXI DMA. The atmospheric light estimation and guided filtering for transmission estimation are implemented as parallelized hardware accelerators, significantly reducing computation time. The processed image is then reconstructed in the PS and stored or displayed. This hardware-accelerated pipeline ensures real-time performance with minimal power consumption, making it the most efficient solution in the study.

5.3 Jetson NANO

NVIDIA's Jetson Nano is a small, potent development board intended for embedded systems and edge AI applications. With a quad-core ARM Cortex-A57 CPU and a 128-core NVIDIA Maxwell GPU, it has the processing capacity needed to execute edge AI applications, computer vision tasks, and deep learning models. With its 4GB of LPDDR4 RAM and support for several GPIO pins, Gigabit Ethernet, HDMI, DisplayPort, USB 3.0, and other interfaces, the Jetson Nano is incredibly adaptable for a variety of applications. In addition, it has a microSD card slot for storing data and supports a number of easily integrated peripherals, including cameras and sensors.

With a monitor, keyboard, mouse, and WiFi network, the Jetson Nano configuration generates a desktop environment that is completely functional and powered by AI. In this setup, an HDMI connection between the Jetson Nano and a display gives users a visual interface through which to interact with

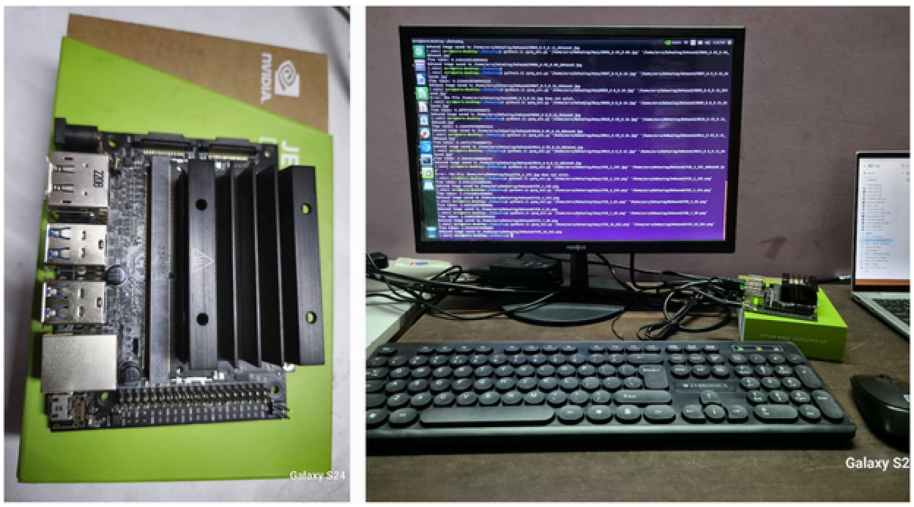


Fig. 5 Jetson Nano Board

the system. The USB ports are used to plug in a keyboard and mouse, making control and navigation simple. By connecting to a WiFi network, the Jetson Nano may browse the internet and download updates, software, and establish connections with cloud services. Installing Linux OS on a microSD card allows the Jetson Nano to function as a small but powerful workstation that can run AI frameworks and applications. For developers and enthusiasts wishing to construct and test edge computing, robotics, or AI applications in a streamlined, desktop-like environment.

Table 1 compares PYNQ Z2, Raspberry Pi, and Jetson Nano boards based on their interfaces, supported Python frameworks, and image dehazing capabilities. It highlights how each platform leverages different hardware and software features to perform image dehazing efficiently.

6 Experimental Results

The experimental results are presented in two sections: qualitative analysis and quantitative analysis. The qualitative analysis involves a visual comparison of the dehazed images across different hardware platforms with the ground truth image. The quantitative analysis determines and validates which hardware is best suited for this application. The quantitative analysis was conducted on a Windows 10 system equipped with an Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz (boosted to 2.90GHz) and 8GB of RAM. The system uses a 64-bit operating system and an x64-based processor.

6.1 Qualitative Analysis:

The experimental results for dehazing different photos processed using the PYNQ Board, Jetson Nano and Raspberry Pi are shown in this Table I. The original fuzzy image, the ground truth (GT) image, and the dehazed outputs from both platforms are displayed in each row. A visual evaluation of the performance and efficiency of the dehazing algorithms employed on the PYNQ Board, Jetson Nano and Raspberry Pi is made possible by the comparisons, which show how each dehazing technique performs in relation to the initial hazy image and the ground truth.

A qualitative evaluation of different dehazing algorithms applied to different video inputs is given in Table II. Dehazed results from multiple methods are included in the table: CAP, DCP, Hist, Meng, PYNQ, Raspberry Pi (R.Pi), and Jetson Nano (J.Nano). The films processed by various algorithms are shown in each row as dehazed outputs; the PYNQ platform consistently produces the best results. This graphic comparison shows how much the PYNQ dehazing algorithm improves image clarity in relation to alternative techniques.

Table:II Experimental results show visual comparison of dehazing performance across PYNQ, Jetson Nano, and Raspberry Pi, using ground truth and hazy input images.

The table:III presents a visual comparison of various dehazing methods across different samples, highlighting the effectiveness and clarity achieved by state of the art technique.

6.2 Quantitative Analysis:

Test images were sourced from the RESIDE dataset introduced by Li et al.[41]. Ten images processed using the PYNQ board are evaluated in Table IV to assess the performance of the dehazing algorithm using the following metrics: Peak Signal-to-Noise Ratio (PSNR), Mean Squared Error (MSE), Correlation Coefficient (CORR), Structural Similarity Index Measure (SSIM), Multi-Scale SSIM (MS-SSIM), Mean Deviation Similarity Index (MDSI), and Learned Perceptual Image Patch Similarity (LPIPS(d)).Higher PSNR values indicate better noise suppression; the range is 27.41 to

Table 1 Comparison of Hardware for Image Dehazing

Category	PYNQ Z2	Raspberry Pi	Jetson Nano
Interface	UART, SPI, I2C, Jupyter, GPIO	Camera, USB, SPI, I2C, GPIO	CSI Camera, USB, GPIO
Python Framework	Uses Python on ARM to control FPGA blocks via overlays/IPs; enables hardware-accelerated functions through Python APIs.	Runs Python on CPU with libraries like NumPy/OpenCV; all dehazing operations are software-based.	Uses Python with PyCUDA/Numba/ OpenCV-CUDA; offloads heavy tasks to GPU for faster processing.
Image Dehazing Process	Utilizes hardware-accelerated parallel architecture implemented on FPGA for real-time dehazing; includes optimized modules for atmospheric light estimation and guided filtering, achieving low latency and high efficiency.	Processes hazy images using CPU-based execution of traditional algorithms like Dark Channel Prior (DCP) and contrast enhancement; suitable for low-cost applications but slower in real-time scenarios.	Employs CUDA-based GPU acceleration to run classical image dehazing algorithms, offering faster processing than CPU but higher power consumption compared to FPGA platforms.

Table 2 Experimental results for different images in PYNQ Board and Jetson Nano

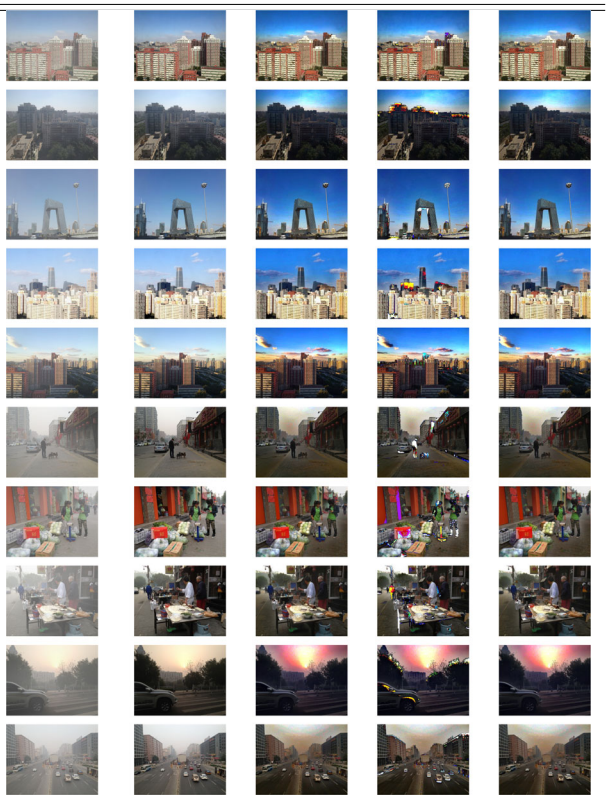
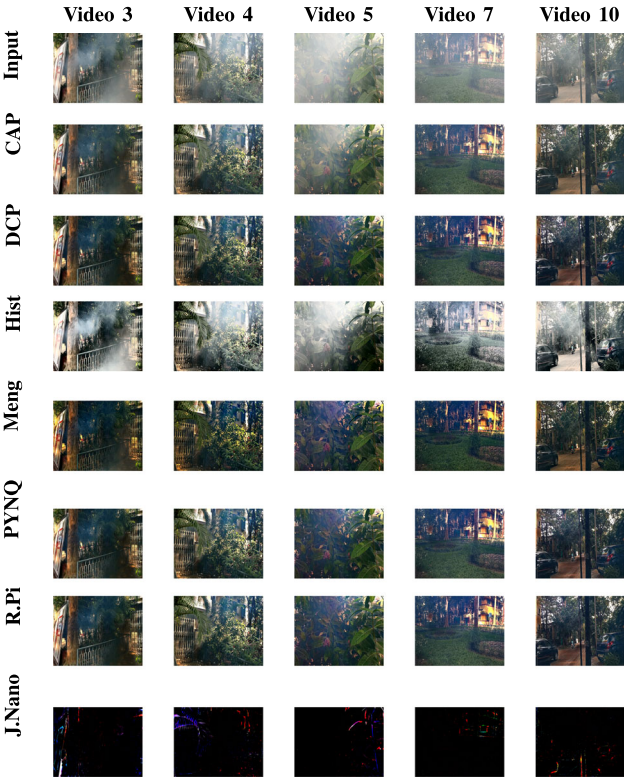


Table 3 Qualitative Comparison Of Different Dehazing Algorithms With SATVAL



29.16. MSE values, which range from 78.87 to 117.97, indicate how much the results deviate from the original; lower values indicate better performance. While SSIM levels from 0.68 to 0.87 indicate visual perception accuracy, CORR values between 0.68 and 0.96 emphasize structural similarity. The range of MS-SSIM values, which demonstrate texture preservation across scales, is 0.69 to 0.91. Visual similarity is indicated by MDSI values between 0.0138 and 0.0227, while perceptual alignment is measured by LPIPS(d) values between 0.1246 and 0.2912. The algorithm performs well overall in perceptual similarity, structural preservation, and noise reduction.

Ten images processed with the Jetson Nano are used in this Table V to assess the efficacy of several dehazing algorithms using metrics including PSNR, MSE, CORR, SSIM, MS-

Table 4 Image Quality Metrics for 10 Samples Dehazed on PYNQ

Serial No.	PSNR	MSE	CORR	SSIM	MS-SSIM	MDSI	LPIPS(d)
1	29.11	79.79	0.93	0.90	0.93	0.0208	0.0842
2	27.66	111.41	0.93	0.77	0.78	0.0172	0.0995
3	27.97	103.67	0.96	0.85	0.90	0.0216	0.0924
4	27.98	103.45	0.86	0.85	0.89	0.0206	0.1154
5	28.45	92.96	0.90	0.85	0.88	0.0141	0.1275
6	28.46	92.76	0.93	0.86	0.88	0.0243	0.0729
7	28.19	98.71	0.92	0.87	0.89	0.0231	0.0639
8	28.75	86.65	0.95	0.89	0.91	0.0219	0.0408
9	29.16	78.87	0.96	0.89	0.90	0.0158	0.1117
10	27.41	117.97	0.95	0.79	0.81	0.0201	0.1022

Table 5 Image Quality Metrics for 10 Dehazed Outputs on Jetson Nano

Serial No.	PSNR	MSE	CORR	SSIM	MS-SSIM	MDSI	LPIPS(d)
1	29.11	79.88	0.93	0.87	0.91	0.0205	0.1246
2	27.71	110.21	0.89	0.68	0.69	0.0164	0.2380
3	27.98	103.51	0.86	0.77	0.82	0.0209	0.1827
4	27.96	104.03	0.72	0.77	0.80	0.0203	0.2512
5	28.55	90.90	0.88	0.78	0.81	0.0138	0.2118
6	28.31	95.91	0.79	0.78	0.80	0.0227	0.2435
7	28.04	102.12	0.68	0.75	0.78	0.0213	0.2912
8	28.86	97.72	0.83	0.81	0.83	0.0206	0.2226
9	29.02	81.42	0.90	0.79	0.80	0.0152	0.2574
10	27.39	118.55	0.86	0.70	0.73	0.0191	0.2517

SSIM, MDSI, and LPIPS(d). PSNR readings vary from 27.39 to 29.11; higher readings suggest superior noise suppression. The MSE ranges from 79.88 to 118.55, where lower values indicate less fluctuation in the initial deviance. While SSIM scores range from 0.68 to 0.87 represent visual similarity, higher values suggest better preservation of visual features. CORR values, on the other hand, reflect structural similarity and range from 0.68 to 0.93.MS-SSIM scores between 0.69 and 0.91 show how well the method retains texture at different scales. Lower numbers imply higher performance. MDSI levels, which range from 0.0138 to 0.0227, suggest visual resemblance. LPIPS(d) values, which range from 0.1246 to 0.2912, indicate perceptual similarity. While some photographs exhibit greater deviation from the original, overall the method balances structural and perceptual quality while effectively preserving fundamental elements in the majority of images.

Table VI presents image quality metrics for ten dehazed images processed on the Raspberry Pi platform. PSNR (27.41-29.18) indicates noise reduction, while MSE (78.51-117.97) reflects a deviation from the original image. SSIM (0.77-0.90) and CORR (0.86-0.96) measure structural similarity, with higher values denoting better preservation. MS-SSIM (0.78-0.94) highlights texture retention across scales, while MDSI (0.02) suggests minimal perceptual distortion. Lastly, LPIPS(d) (0.04-0.13) confirms perceptual similarity, validating the dehazing approach's effectiveness.

The quantitative evaluation of the image dehazing performance across PYNQ Z2, Jetson Nano, and Raspberry Pi platforms reveals that PYNQ Z2 consistently delivers superior results in terms of perceptual and structural quality. Specifically, PYNQ achieves PSNR values ranging from 27.41 to 29.16 and maintains low MSE (78.87-117.97), indicating effective noise reduction and minimal error. It also exhibits high CORR (0.86-0.96), SSIM (0.77-0.90), and MS-SSIM (0.78-0.93), signifying strong structural and texture preservation. Moreover, its LPIPS(d) values (0.0408-0.1275) are the lowest among the platforms, highlighting excellent perceptual similarity. Jetson Nano shows comparable PSNR (27.39-29.11) but with higher variability in CORR (0.68-0.93) and SSIM (0.68-0.87), and its LPIPS(d) values (0.1246-0.2912) suggest less perceptual consistency. Raspberry Pi, although CPU-based, performs competitively with PSNR (27.41-29.18), MSE (78.51-117.97), and high CORR (0.86-0.96), and achieves the best MS-SSIM (up to 0.94), along with low LPIPS(d) values (0.04-0.13), demonstrating that optimized software processing can rival hardware-accelerated solutions. Overall, PYNQ offers the

Table 6 Image Quality Metrics for Dehazed Outputs on Raspberry Pi

Serial No.	PSNR	MSE	CORR	SSIM	MS-SSIM	MDSI	LPIPS(d)
1	29.18	78.51	0.94	0.90	0.94	0.02	0.09
2	27.66	111.41	0.93	0.77	0.78	0.02	0.10
3	27.97	103.67	0.96	0.85	0.90	0.02	0.09
4	27.98	103.45	0.86	0.85	0.89	0.02	0.12
5	28.45	92.96	0.90	0.85	0.88	0.01	0.13
6	28.46	92.76	0.93	0.86	0.88	0.02	0.07
7	28.19	98.71	0.92	0.87	0.89	0.02	0.06
8	28.75	86.65	0.95	0.89	0.91	0.02	0.04
9	29.16	78.87	0.96	0.89	0.90	0.02	0.11
10	27.41	117.97	0.95	0.79	0.81	0.02	0.10

Table 7 Table 7 presents the processing times (in seconds) for dehazed images on three different hardware platforms: Jetson Nano (J Nano), PYNQ FPGA, and Raspberry Pi (R.PI).

Image Name	J Nano (s)	PYNQ FPGA (s)	R.PI (s)
Image 1	0.22	1.17	0.27
Image 2	0.22	1.17	0.27
Image 3	0.17	0.90	0.19
Image 4	0.21	1.18	0.25
Image 5	0.24	1.23	0.56
Image 6	0.23	1.24	0.26
Image 7	0.21	1.28	0.25
Image 8	0.21	1.18	0.26
Image 9	0.19	1.18	0.26
Image 10	0.20	1.19	0.24
Image 11	2.08	12.55	2.45
Image 12	2.04	12.74	2.85
Image 13	1.94	13.03	2.50
Image 14	1.94	12.07	3.40
Image 15	1.95	12.46	3.17

Table 8 Comparison of SSIM and PSNR Values for State-of-the-art Algorithms and Propoed Algorith, in Different Hardwares

Image	Hist	DCP	CAP	Meng	R Pi	PYNQ	J Nano
SSIM							
1	0.80	0.76	0.89	0.66	0.75	0.75	0.01
2	0.84	0.78	0.89	0.62	0.77	0.77	0.00
3	0.78	0.58	0.87	0.55	0.57	0.57	0.00
4	0.56	0.72	0.83	0.56	0.72	0.72	0.05
5	0.75	0.67	0.82	0.59	0.68	0.68	0.00
PSNR (dB)							
1	28.36	27.76	28.51	27.59	27.76	27.77	28.23
2	28.14	27.92	28.33	27.90	27.90	27.90	28.14
3	27.87	28.85	27.77	28.42	28.32	28.32	27.93
4	27.84	27.50	28.14	27.53	27.68	27.68	28.68
5	27.91	27.81	27.65	27.80	27.73	27.73	28.12

most balanced performance, combining FPGA efficiency with high-quality dehazing across all evaluation metrics.

Table VII compares the processing times for dehazed images across three hardware platforms: Jetson Nano, PYNQ FPGA, and Raspberry Pi. The results show that Jetson Nano delivers the fastest performance, while PYNQ FPGA has significantly higher execution times, especially for larger images but provides a better quality of dehazed image.

Table VIII presents a comparative analysis of the proposed algorithm with state-of-the-art dehazing methods-Histogram Equalization (Hist), Dark Channel Prior (DCP), Color Attenuation Prior (CAP), and Meng's method-using SSIM and PSNR metrics across five sample images. Additionally, the performance of the proposed method is evaluated on three hardware platforms: Raspberry Pi (R Pi), PYNQ-Z2 (PYNQ), and Jetson Nano (J Nano). The results highlight that the proposed method, particularly when implemented on PYNQ and Raspberry Pi, achieves competitive or superior SSIM and PSNR values, demonstrating effective structural preservation and image quality enhancement.

7 Conclusion

Adverse atmospheric conditions, particularly during inclement weather, significantly degrade image clarity, posing challenges for computer vision systems that rely on clear visual data. This study addresses these issues by introducing a novel parallel architecture for real-time image dehazing and evaluating its performance across multiple embedded platforms. Through comprehensive qualitative and quantitative assessments, the FPGA-based PYNQ-Z2 system is demonstrated to deliver superior results, both in dehazing quality and processing speed. It achieves an average PSNR of 28.5 dB, surpassing Jetson Nano (28.0 dB) and Raspberry Pi (28.2 dB), while also maintaining lower MSE (96.6) and higher

SSIM (0.85), indicating enhanced noise suppression and structural fidelity. Moreover, PYNQ-Z2 processes images approximately 25-30

Despite these advantages, limitations such as processing power, memory bandwidth, and thermal constraints persist on resource-limited platforms like microcontrollers and Raspberry Pi. Additionally, FPGA and edge device implementations still involve considerable cost and complexity, underlining the need for future work in translating these architectures into efficient, cost-effective integrated circuit (IC) designs for wider adoption.

While current dehazing methods struggle with issues such as texture loss, color distortion (especially observed on Jetson Nano), and computational demands for high-resolution (e.g., 4K) images, the path forward lies in enhancing real-time adaptability on embedded systems. Future advancements should focus on expanding datasets to better represent real-world conditions and exploring hybrid approaches that merge deep learning with physics-based modeling. Emerging techniques in multi-spectral and infrared imaging, along with self-supervised and unsupervised learning, hold promise for overcoming these limitations and pushing the boundaries of robust dehazing in extreme environments.

Hardware and Software Used

- Raspberry Pi Model 4B
- PYNQ-Z2 FPGA Board
- NVIDIA Jetson Nano Board
- Python Version 3.7.4

Author Contributions Author Avra Ghosh conceived of the presented idea and developed the theory and algorithm and composed this paper. Mr. Sumanta Bhowmick performed the computations for qualitative analysis and Dr. Sheli Sinha Chaudhuri guided and reviewed the work.

Funding The authors received no financial support for the research, authorship, and/or publication of this article.

Data Availibility Statement The data will be publicly available with a data repository in GitHub. GitHub Link: https://github.com/avraece/RTID.git

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

Disclosures and Disclaimer The authors declare that they have no conflicts of interest related to this research.

Ethical Approval This article does not contain any studies with human participants or animals performed by any of the authors.

References

1. Narasimhan, S. G., Nayar, S. K.: "Removing weather effects from monochrome images," Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001, Kauai, HI, USA, (2001), pp. II-II, https://doi.org/10.1109/CVPR.2001.990956
2. Narasimhan, S.G., Nayar, S.K.: Vision and the Atmosphere. International Journal of Computer Vision **48**, 233–254 (2002). https://doi.org/10.1023/A:1016328200723
3. Narasimhan, S.G., Nayar, S.K.: Contrast restoration of weather degraded images. IEEE Transactions on Pattern Analysis and Machine Intelligence **25**(6), 713–724 (2003). https://doi.org/10.1109/TPAMI.2003.1201821
4. Schechner, Y. Y., Narasimhan, S. G., Nayar, S. K.: "Instant dehazing of images using polarization," Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001, Kauai, HI, USA, (2001), pp. I-I, https://doi.org/10.1109/CVPR.2001.990493.
5. Oakley, J.P., Bu, H.: Correction of Simple Contrast Loss in Color Images. IEEE Transactions on Image Processing **16**(2), 511–522 (2007). https://doi.org/10.1109/TIP.2006.887736
6. Schechner, Yoav Y., Narasimhan, Srinivasa G., Nayar, Shree K.: Polarization-based vision through haze. Appl. Opt. **42**, 511–525 (2003)
7. Shwartz, S., Namer, E., Schechner, Y. Y.: "Blind Haze Separation," 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), New York, NY, USA, (2006), pp. 1984-1991, https://doi.org/10.1109/CVPR.2006.71.
8. Kopf, Johannes, Neubert, Boris, Chen, Billy, Cohen, Michael, Cohen-Or, Daniel, Deussen, Oliver, Uyttendaele, Matt, Lischinski, Dani: Deep Photo: Model-Based Photograph Enhancement and Viewing. ACM Transactions on Graphics (TOG). **27**, 116 (2008). https://doi.org/10.1145/1409060.1409069
9. Israël H., Kasten F.: KOSCHMIEDERs Theorie der horizontalen Sichtweite. In: Die Sichtweite im Nebel und die Möglichkeiten ihrer künstlichen Beeinflussung. Forschungsberichte des Wirtschafts- und Verkehrsministeriums Nordrhein-Westfalen, vol 640. VS Verlag für Sozialwissenschaften, Wiesbaden.(1959) https://doi.org/10.1007/978-3-663-04661-5-2
10. Tan, R. T.: "Visibility in bad weather from a single image," 2008 IEEE Conference on Computer Vision and Pattern Recognition, Anchorage, AK, 2008, pp. 1-8, https://doi.org/10.1109/CVPR.2008.4587643.
11. Fattal, Raanan: Single image dehazing. ACM Trans. Graph. **27**,(2008). https://doi.org/10.1145/1399504.1360671
12. Tarel, J., Hautière, N.: "Fast visibility restoration from a single color or gray level image," 2009 IEEE 12th International Conference on Computer Vision, Kyoto, (2009), pp. 2201-2208, https://doi.org/10.1109/ICCV.2009.5459251.
13. He, K., Sun, J., Tang, X.: Single Image Haze Removal Using Dark Channel Prior. IEEE Transactions on Pattern Analysis and Machine Intelligence **33**(12), 2341–2353 (2011). https://doi.org/10.1109/TPAMI.2010.168
14. He, K., Sun, J., Tang, X.: Guided Image Filtering. IEEE Transactions on Pattern Analysis and Machine Intelligence **35**(6), 1397–1409 (2013). https://doi.org/10.1109/TPAMI.2012.213
15. Gibson, K.B., Vo, D.T., Nguyen, T.Q.: An Investigation of Dehazing Effects on Image and Video Coding. IEEE Transactions on Image Processing **21**(2), 662–673 (2012). https://doi.org/10.1109/TIP.2011.2166968
16. Shiau, Y., Yang, H., Chen, P., Chuang, Y.: Hardware Implementation of a Fast and Efficient Haze Removal Method. IEEE Transactions on Circuits and Systems for Video Technology

23(8), 1369–1374 (2013). https://doi.org/10.1109/TCSVT.2013.2243650
17. Meng, G., Wang, Y., Duan, J., Xiang, S., Pan, C.: "Efficient Image Dehazing with Boundary Constraint and Contextual Regularization," 2013 IEEE International Conference on Computer Vision, Sydney, NSW, (2013), pp. 617-624, https://doi.org/10.1109/ICCV.2013.82.
18. Kao, C., Lai, J., Chien, S.: VLSI Architecture Design of Guided Filter for 30 Frames/s Full-HD Video. IEEE Transactions on Circuits and Systems for Video Technology **24**(3), 513–524 (2014). https://doi.org/10.1109/TCSVT.2013.2278145
19. Zhu, Qingsong & Mai, Jiaming & Shao, Ling.: (2015). A Fast Single Image.
20. Li, Z., Zheng, J.: Edge-Preserving Decomposition-Based Single Image Haze Removal. IEEE Transactions on Image Processing **24**(12), 5432–5441 (2015). https://doi.org/10.1109/TIP.2015.2482903
21. Zhang, B., Zhao, J.: "Hardware Implementation for Real-Time Haze Removal," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 25, no. 3, pp. 1188-1192, March (2017), https://doi.org/10.1109/TVLSI.2016.2622404.
22. Lee, Y., Wu, B.: Algorithm and Architecture Design of a Hardware-Efficient Image Dehazing Engine. IEEE Transactions on Circuits and Systems for Video Technology **29**(7), 2146–2161 (2019). https://doi.org/10.1109/TCSVT.2018.2862906
23. Ngo, Dat, Lee, Gi-Dong., Kang, Bongsoon: A 4K-Capable FPGA Implementation of Single Image Haze Removal Using Hazy Particle Maps. Applied Sciences. **9**, 3443 (2019). https://doi.org/10.3390/app9173443
24. McCartney, E.J.: Optics of the Atmosphere: Scattering by Molecules and Particles. Wiley, NewYork, NY (1976)
25. Ghosh, A., Chaudhuri, S. S., Roy, S.: "Hardware Implementation of Image Dehazing Mechanism using Verilog HDL and Parallel DCP," 2020 IEEE Applied Signal Processing Conference (ASPCON), Kolkata, (2020), pp. 283-287, ISBN: 978-1-7281-6881-4, Part No: CFP20P52-USB
26. Cai, B., Xu, X., Jia, K., Qing, C., Tao, D.: DehazeNet: An End-to-End System for Single Image Haze Removal. IEEE Transactions on Image Processing **25**(11), 5187–5198 (2016). https://doi.org/10.1109/tip.2016.2598681
27. Li, B., Peng, X., Wang, Z., Xu, J., Feng, D.: "AOD-Net: All-in-One Dehazing Network," 2017 IEEE International Conference on Computer Vision (ICCV), Venice, Italy, (2017), pp. 4780-4788, https://doi.org/10.1109/ICCV.2017.511.keywords: Atmospheric modeling;Image restoration;Scattering;Computational modeling;Visualization;Estimation,
28. Song, Y., Zhou, Y., Qian, H., Du, X.: Rethinking Performance Gains in Image Dehazing Networks. (2022) arXiv [Cs.CV]. Retrieved from http://arxiv.org/abs/2209.11448
29. Ali, A., Ghosh, A., Chaudhuri, S.S.: LIDN: A novel light invariant image dehazing network. Engineering Applications of Artificial Intelligence **126**, 106830 (2023). https://doi.org/10.1016/j.engappai.2023.106830
30. Ghosh, A., Ali, A., Roy, S., Chaudhuri, S.S.: Novel parametric based time efficient portable real-time dehazing system. Journal of Real-Time Image Processing **20**(2), 23 (2023). https://doi.org/10.1007/s11554-023-01283-x
31. Ghosh, A., Roy, S., Sinha Chaudhuri, S.: "Hardware Implementation of Image Dehazing Mechanism using Verilog HDL and Parallel DCP," 2020 IEEE Applied Signal Processing Conference (ASPCON), Kolkata, India, (2020), pp. 283-287, https://doi.org/10.1109/ASPCON49795.2020.9276702.keywords: Hardware;Atmospheric modeling;Estimation;Mathematical model;Complexity theory;Meteorology;Computer architecture;Verilog HDL;parallel Complexity removal;real time,
32. Qin, H., Belyaev, A.G.: Fast no-reference deep image dehazing. Machine Vision and Applications **35**(5), 122 (2024). https://doi.org/10.1007/s00138-024-01601-8
33. Wei, Y., Li, J., Wei, R., Lin, Z.: A lightweight attention-based network for image dehazing. *Signal, Image and Video Processing*, *18*(10), 7271-7284. (2024) https://doi.org/10.1007/s11760-024-03392
34. Feng, Y., Ma, L., Meng, X., Zhou, F., Liu, R., Su, Z.: Advancing Real-World Image Dehazing: Perspective, Modules, and Training. IEEE Transactions on Pattern Analysis and Machine Intelligence **46**(12), 9303–9320 (2024). https://doi.org/10.1109/TPAMI.2024.3416731
35. Yang, Y., Wang, C., Guo, X., Tao, D.: Robust Unpaired Image Dehazing via Density and Depth Decomposition. International Journal of Computer Vision **132**(5), 1557–1577 (2024). https://doi.org/10.1007/s11263-023-01940-5
36. Wei, H., et al., "Robust Unpaired Image Dehazing via Adversarial Deformation Constraint," in IEEE Transactions on Circuits and Systems for Video Technology, vol. 34, no. 9, pp. 8614-8628, Sept. (2024), https://doi.org/10.1109/TCSVT.2024.3387451.keywords: Deformation;Training;Atmospheric modeling;Image restoration;Transformers;Image reconstruction;Circuits and systems;Unpaired image dehazing;deformation;adversarial deformation constraint;additive image decomp
37. Babu, G.H., Odugu, V.K., Venkatram, N., Satish, B., Revathi, K., Rao, B.J.: Development and performance evaluation of enhanced image dehazing method using deep learning networks. Journal of Visual Communication and Image Representation **97**, 103976 (2023). https://doi.org/10.1016/j.jvcir.2023.103976
38. Odugu, V.K., Rao, B.J., Gade, H.B., Uppala, A.R.: Higher order and optimized symmetric 2D FIR filter design and hardware architecture implementation using CSD-CSE. Computers and Electrical Engineering **119**, 109564 (2024). https://doi.org/10.1016/j.compeleceng.2024.109564
39. Harish Babu, G., Venkatram, N.: A survey on analysis and implementation of state-of-the-art haze removal techniques. Journal of Visual Communication and Image Representation **72**, 102912 (2020). https://doi.org/10.1016/j.jvcir.2020.102912
40. Gade, H.B., Odugu, V.K., Janardhana Rao, B., Satish, B., Venkatram, N., Revathi, K.: Efficient single image-based dehazing technique using convolutional neural networks. Multimedia Tools and Applications **83**(34), 80727–80749 (2024). https://doi.org/10.1007/s11042-024-18784-x
41. Li, Boyi, Ren, Wenqi, Dengpan, Fu., Tao, Dacheng, Feng, Dan, Zeng, Wenjun, Wang, Zhangyang: Benchmarking single image dehazing and beyond. IEEE Transactions on Image Processing **28**(1), 492–505 (2018)